MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING

B.tech (CE)
Semester: III
Course: Data Structure and Algorithms
Academic Year: 2026-27

Presented by: *Ishani Kansal (U058) - 70022500643*
Kirti Purohit(U065) - 70022500111
Sania Shroff(U106) - 70022500355
Avantika Dalal(U034) - 70022500354

Index

- *Introduction*
- *Problem Statement*
- *Objectives*
- *Data Structure used*
- *Data Structures Used — Linked List & Stack*
- *Circular Queue & Core Operations*
- *Flow of the code*
- *Output*
- *Conclusion & Future Scope*

Introduction

A music playlist contains a collection of songs that users can play, navigate, and manage.
The Music Playlist Manager is a C-based application that demonstrates how different Data Structures can be combined to implement real-world music-player operations.

The project uses:

- ***Doubly Linked List*** for storing and navigating playlist songs.
- ***Stack*** for maintaining the history of played songs.
- ***Queue*** for storing songs that are scheduled to play next.

Problem Statement

- ***Inflexible Song Navigation***: Standard linear arrays cannot natively navigate backward and forward efficiently without costly index tracking or memory reallocation.

- ***Lack of Dynamic Up Next Queueing:*** Conventional audio lists struggle to temporarily insert user-requested tracks without cluttering or altering the primary sequence order.

- ***Loss of Playback History:*** Simple pointers lose track of previously visited songs, making immediate, accurate step-back actions difficult to implement.

Objectives

Main objectives are:

1. To implement a practical application using DSA.
2. To understand the working of a Doubly Linked List.
3. To implement Stack using an array.
4. To implement Queue using an array.
5. To demonstrate interaction between multiple data structures.
6. To provide next/previous music navigation.
7. To maintain played-song history.
8. To implement an interactive menu-driven C program

Data Structures Used

Data Structure	Application in Project
Doubly Linked List	Stores playlist songs
Stack	Stores history of played songs.
Queue	Stores songs waiting to be played.

->Doubly Linked List : allows movement in both directions.
->Stack : follows **LIFO** (Last In, First Out), which is useful for recent playback history.
->Queue : follows **FIFO** (First In, First Out), which is suitable for "up next" songs

Data Structures Used — Linked List & Stack

Doubly Linked List (Main Playlist):

- Provides bi-directional traversal(prev / next) for moving forward and backward.
- Dynamic allocation without fixed size constraints.
- Structure: struct Song { char name[50]; struct Song *prev, *next; };
- Tracked by global head and current pointers.

Array-Based Stack (Playback History):

- LIFO (Last-In, First-Out) structure to store history of played tracks.
- Bounded array historyStack[MAX] with index top.
- Core operations: push() adds played song, pop() retrieves last played song, showHistory() displays list

Circular Queue & Core Operations

Circular Queue ("Up Next" Queue):

- FIFO (First-In, First-Out) structure to temporarily insert tracks ahead of regular sequence.
- Uses circular array indexing: (rear + 1) % MAX and (front + 1) % MAX.
- Core operations: enqueue() adds song to queue, dequeue() pops next track, showQueue() lists upcoming queued tracks.

Key Operations Summary:

- **addSong():** Appends node dynamically to the end of the Doubly Linked List.
- **showPlaylist():** Traverses list from head and flags active track (-> Song (playing)).

Flow of the code

Data Structure Flow:

Playlist → Doubly Linked List

↓

Play Next → Push current song into Stack

↓

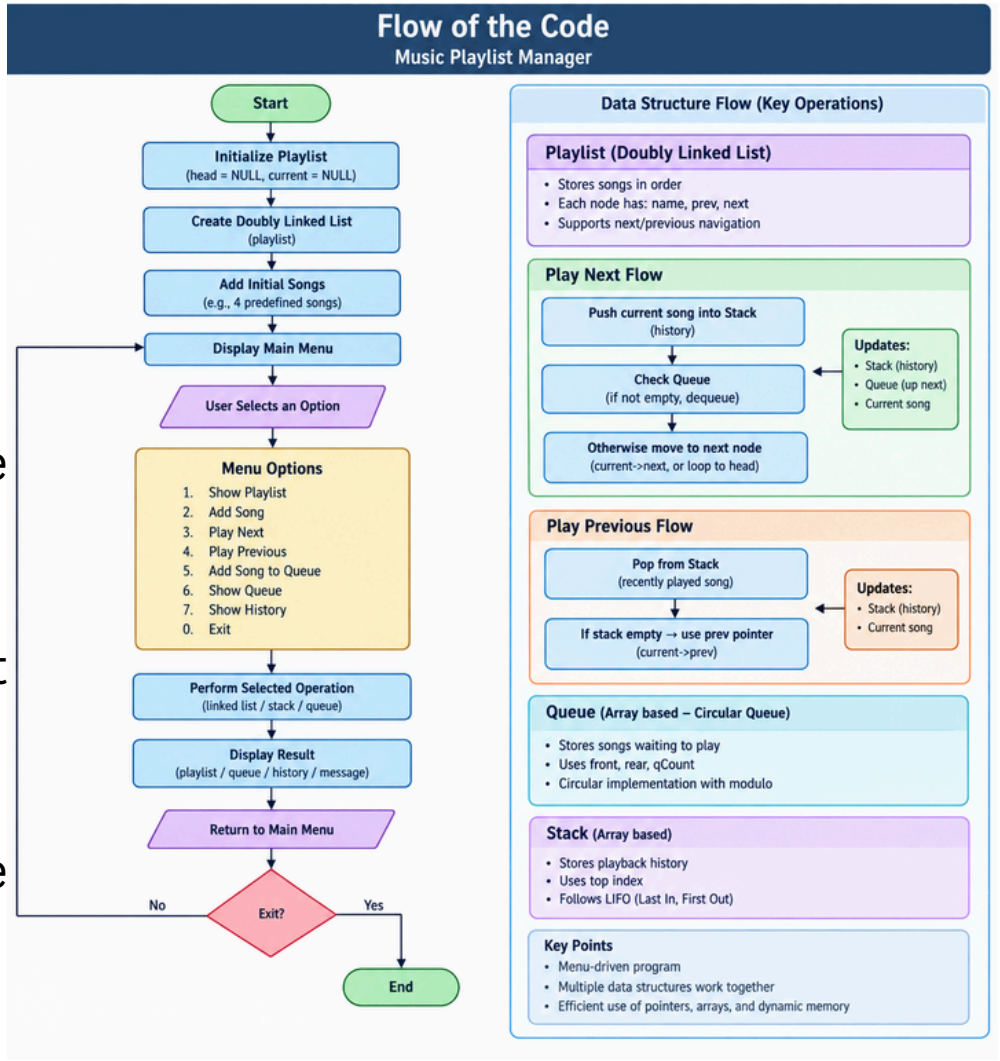
Check Queue → Dequeue if songs are waiting

↓

Otherwise → Move to next Linked List node

↓

Play Previous → Pop from Stack / Move using prev pointer



Output

```
 Music Playlist Manager
1. Show Playlist
2. Add Song
3. Play Next
4. Play Previous
5. Add Song to Queue
6. Show Queue
7. Show History
0. Exit
Enter choice: 1
-> Blinding Lights (playing)
Levitating
Peaches
As It Was

 Music Playlist Manager
1. Show Playlist
2. Add Song
3. Play Next
4. Play Previous
5. Add Song to Queue
6. Show Queue
7. Show History
0. Exit
Enter choice: 2
Enter song name: Love Story
```

```
 Music Playlist Manager
1. Show Playlist
2. Add Song
3. Play Next
4. Play Previous
5. Add Song to Queue
6. Show Queue
7. Show History
0. Exit
Enter choice: 3
Now playing: Levitating

 Music Playlist Manager
1. Show Playlist
2. Add Song
3. Play Next
4. Play Previous
5. Add Song to Queue
6. Show Queue
7. Show History
0. Exit
Enter choice: 4
Now playing: Blinding Lights
```

```
 Music Playlist Manager
1. Show Playlist
2. Add Song
3. Play Next
4. Play Previous
5. Add Song to Queue
6. Show Queue
7. Show History
0. Exit
Enter choice: 5
Enter song name to queue: Stay

 Music Playlist Manager
1. Show Playlist
2. Add Song
3. Play Next
4. Play Previous
5. Add Song to Queue
6. Show Queue
7. Show History
0. Exit
Enter choice: 6
Stay
```

```
 Music Playlist Manager
1. Show Playlist
2. Add Song
3. Play Next
4. Play Previous
5. Add Song to Queue
6. Show Queue
7. Show History
0. Exit
Enter choice: 0
Goodbye!

Process returned 0 (0x0)   execution time : 60.853 s
Press any key to continue.
```

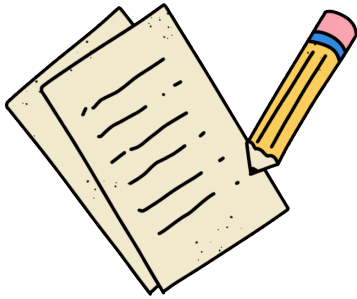
Conclusion & Future Scope

Conclusion

- The project demonstrates the practical use of Doubly Linked List, Stack, and Queue.
- These structures work together to manage playlists, playback history, and upcoming songs.
- It shows how DSA concepts can be applied to a real-world application.

Future Scope

- Add song search and deletion.
- Add shuffle and repeat options.
- Save playlists using file handling.
- Support multiple playlists.



Thank you



Questions & Discussion